Kotlin Backend Developer.Professional

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Тестирование проекта

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Маршрут вебинара

- Бизнес-требования
- Разработка тестов, TDD, MDD
- Тестовые фреймворки
- Интеграционные и сквозные тесты
- Рефлексия

Команда курса

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

Цели вебинара

После занятия вы сможете

1. Проектировать тесты
2. Работать в парадигме TDD
3. Использовать различные тестовые фреймворки и инструменты
4. Разрабатывать сквозные и интеграционные тесты

Смысл

Зачем вам это уметь

1. Для создания качественных проектов, для понимания полноты и законченности требований, для оценки возможных рисков сбоя
2. Различные фреймворки и инструменты позволяют сделать тестирование более качественным и удобным
3. Использование TDD помогает проектировать решение задачи, MDD позволяет анализировать качество ПО в реальном времени

Контроль качества

Контроль качества Quality Assurance

<u>Базовые принципы</u>:

1. Инцидент -

Контроль качества Quality Assurance

<u>Базовые принципы</u>:

1. Инцидент - событие, при котором невозможно исполнение компанией своих договорных обязательств перед клиентами
2. Назначение контроля качества:
   - успешно пройти Приемо-Сдаточные Испытания;
   - снизить число и вероятность инцидентов;
   - контролировать изменения постановки и соответствие реализации.

Контроль качества Quality Assurance

<u>Общие сведения</u>:

1. Любое новое изделие проходит испытания (англ. Testing) перед выходом в пром.эксплуатацию/серию.
2. Любое серийное изделие проходит контроль качества
3. Во время эксплуатации многие системы оборудуются подсистемами мониторинга
4. Для получения сертификатов, аттестаций и лицензий выполняется тестирование изделия или ПО

Наглядный пример

Saturn V static test stand

Лунная гонка

Авария “лунной” ракеты Н-1 (СССР)

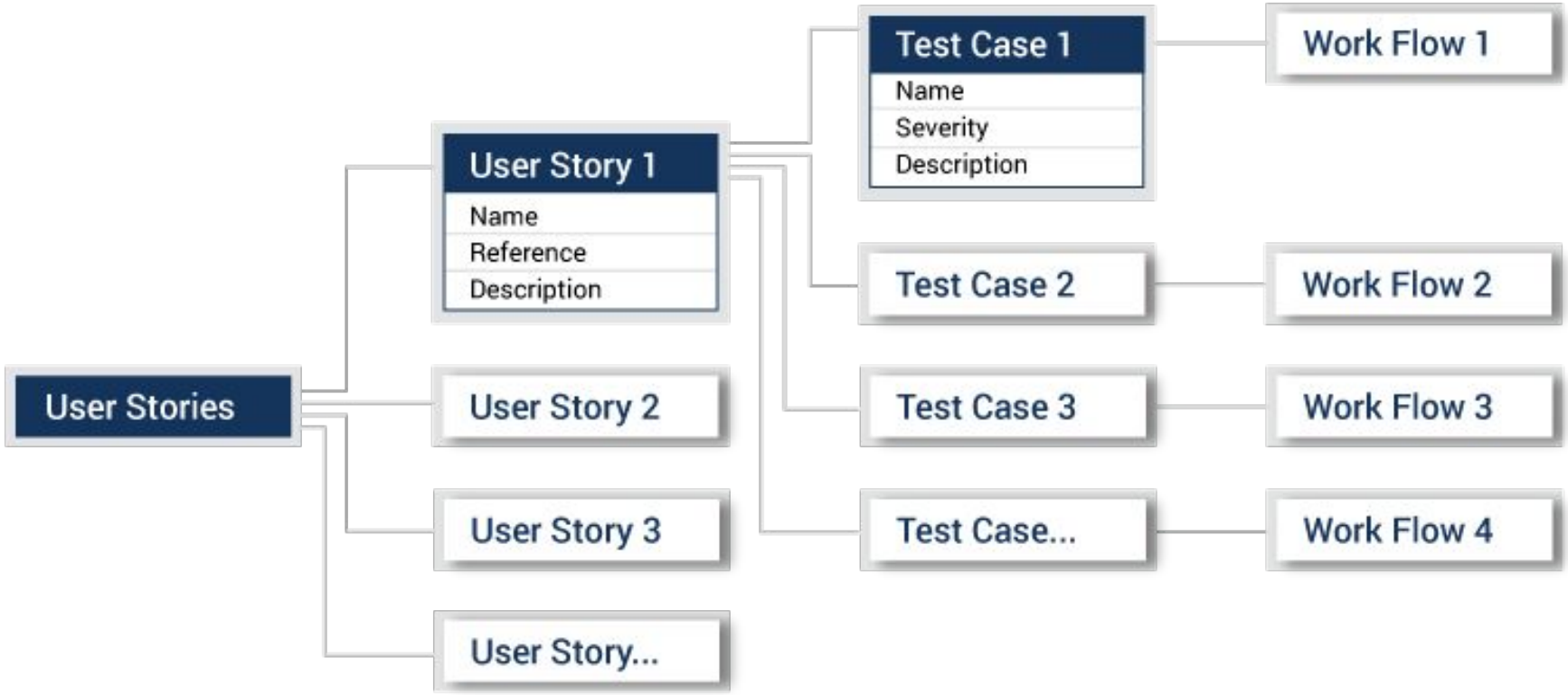
наземные стенды для динамических и огневых испытаний всего носителя или первой ступени в сборе не строились
https://ru.wikipedia.org/wiki/H-1

Значит!!!

Любая разработка
должна начинаеться с
тестового стенда

Бизнес-требования к тестированию

Зачем нужно тестирование



Любые требования (бизнес/функц./ нефункц.) ➔ Сценарии / Критерии приемки ➔ Приемочные тесты

Любой приемочный тест должен закрывать сценарий требования.
Если нет требования, значит нужно добавить через б.аналитиков

Управление тестовыми кейсами

Особенности управления:

1. Огромное количество тестовых кейсов
2. Постановки постоянно меняются

Системы управления требованиями / тестированием:

1. https://habr.com/ru/post/461205/
2. https://blog.jetbrains.com/ru/youtrack/2020/11/make-it-workflow-part-13-supporting-test-management-scenarios/
3. Allure

Req No	Req Desc	Testcase ID	Status
123	Login to the application	TC01,TC02,TC03	TC01-Pass TC02-Pass
345	Ticket Creation	TC04,TC05,TC06, TC07,TC08,TC09 TC010	TC04-Pass TC05-Pass TC06-Pass TC06-Fail TC07-No Run
456	Search Ticket	TC011,TC012, TC013,TC014	TC011-Pass TC012-Fail TC013-Pass TC014-No Run

Traceability matrix
Матрица трассируемости

Тестирование

Что тестируем

<u>Особенности тестирования</u>:

1. Тестируем приемочные сценарии
2. Все внутренние функции
3. Помимо положительного теста, необходимо предусмотреть все нештатные ситуации

Как тестируем

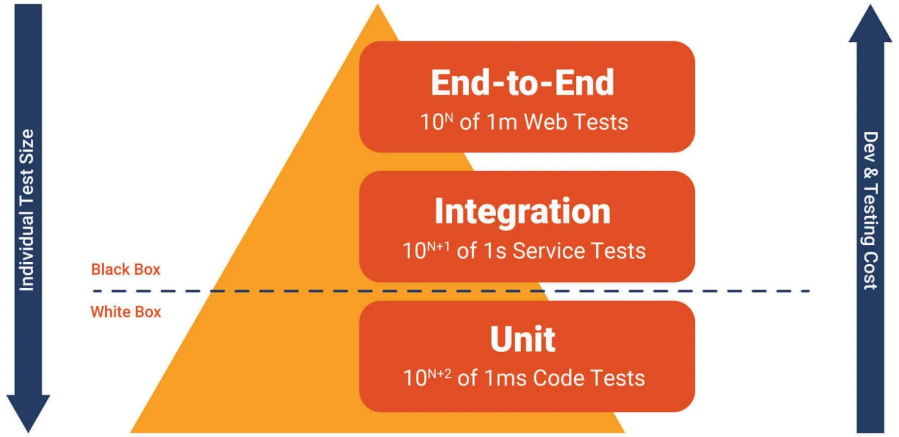
Особенности тестирования:

1. Переменные с возможными значениями
   a. A: Null, True, False
   b. B: Null, 0..1000
2. Тестовые кейсы:
   a. A: Null, True, False, Other
   b. B: Null, -1, 0, 500, 1000, 1001, 100_000
3. Всего: 4*7 тестовых кейсов - полный набор, либо 4+7 - сокращенный

Дешевле протестировать 10 функций с 1 параметром каждая, чем 1 функцию с 10 параметрами.

Разбивка кода на мелкие тестируемые функции

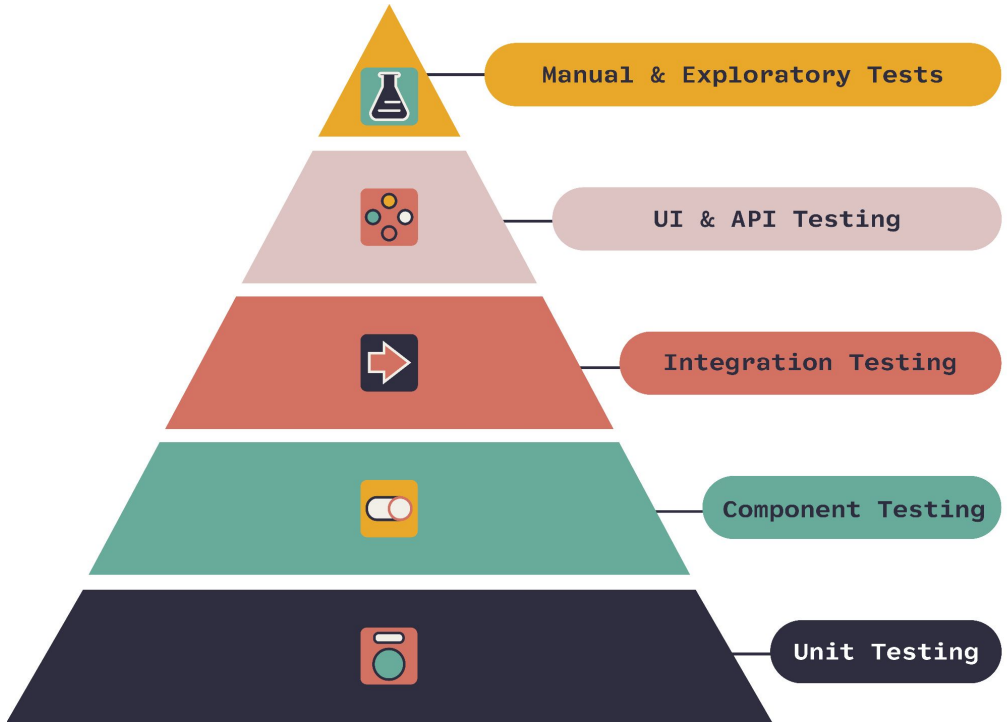
Уровни тестирования



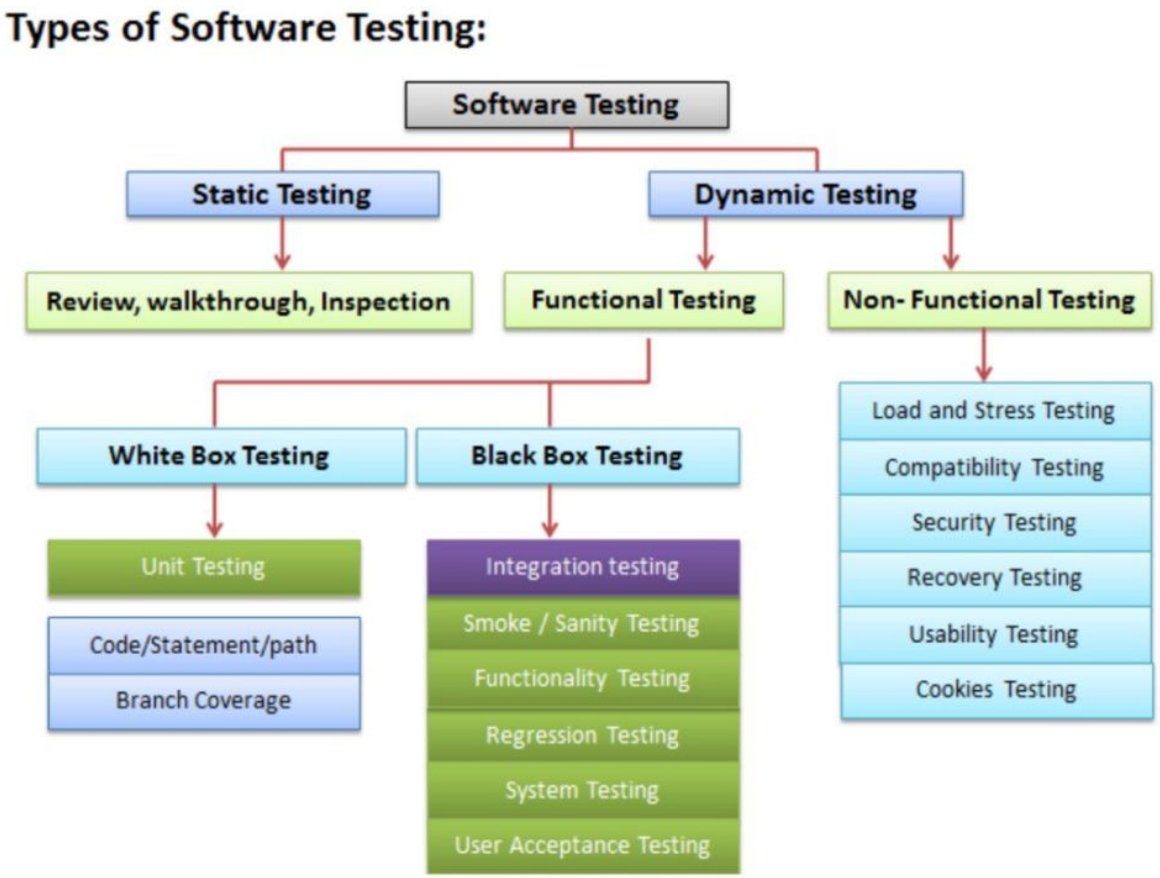
Уровни тестирования:

1. Unit/Модульное - отдельных функций и компонентов
2. Integration/Интеграционное - взаимодействий
3. Сквозное/End-To-End - всей системы

https://habr.com/ru/post/279535/



Виды тестирования



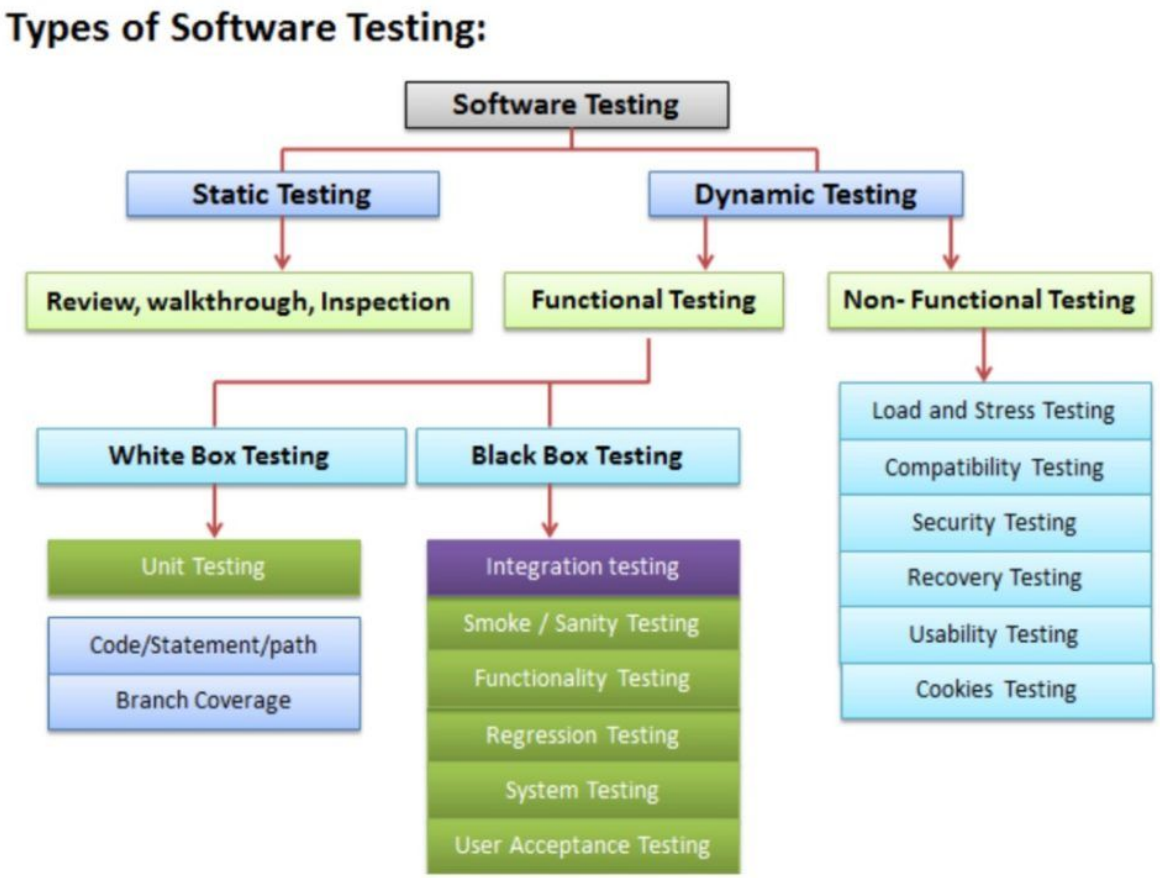
<u>Функциональные</u>:

1. Функциональное
2. Тестирование безопасности
3. Тестирование взаимодействия

Виды тестирования

<u>Нефункциональные</u>:

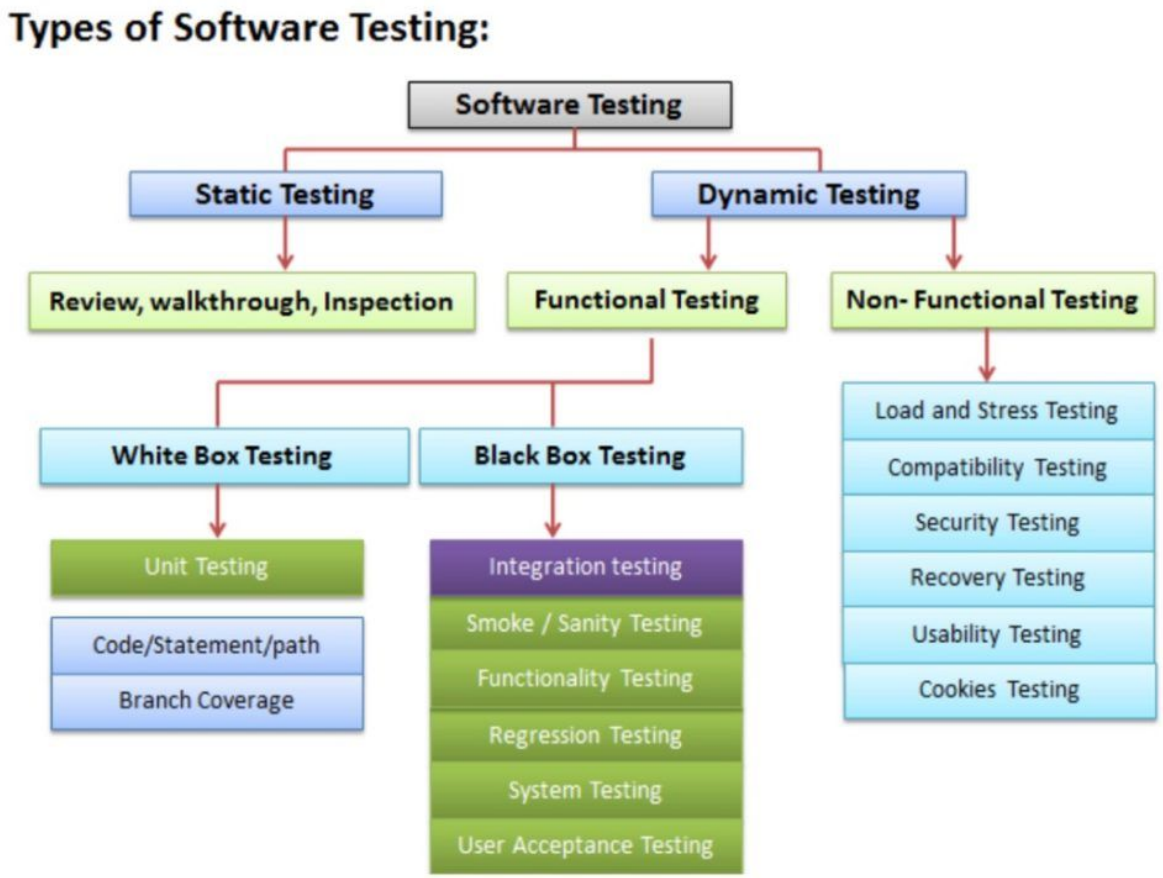
1. Тестирование производительности:
   a. нагрузочное
   b. стрессовое
   c. стабильности или надежности
   d. объемное (много данных)
2. Тестирование установки
3. Тестирование удобства пользования
4. Тестирование на отказ и восстановление
5. Конфигурационное тестирование



Виды тестирования

<u>Тестирование обновлений</u>:

1. Дымовое тестирование (включение - дым)
2. Регрессионное тестирование (новые баги в старом коде)
3. Тестирование сборки
4. Санитарное тестирование или проверка согласованности/исправности



LIVE

проектирование тестов

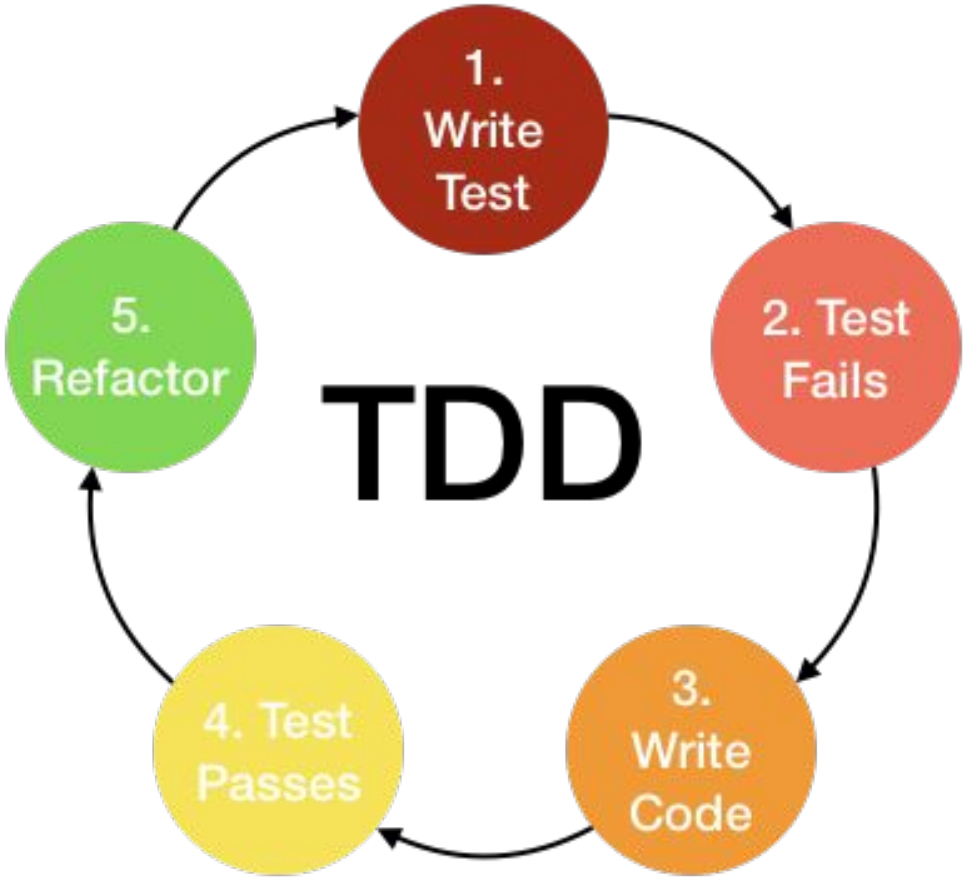
Подходы к тестированию

TDD

Test Driven Development:

1. Тест появляется ДО начала разработки
2. Тест “красный”
3. Разработка внедряет логику
4. Добиваемся “зеленого” теста
5. Оптимизируем/рефакторим код
6. Поддерживаем “зеленый” тест

Постановка задачи в виде теста

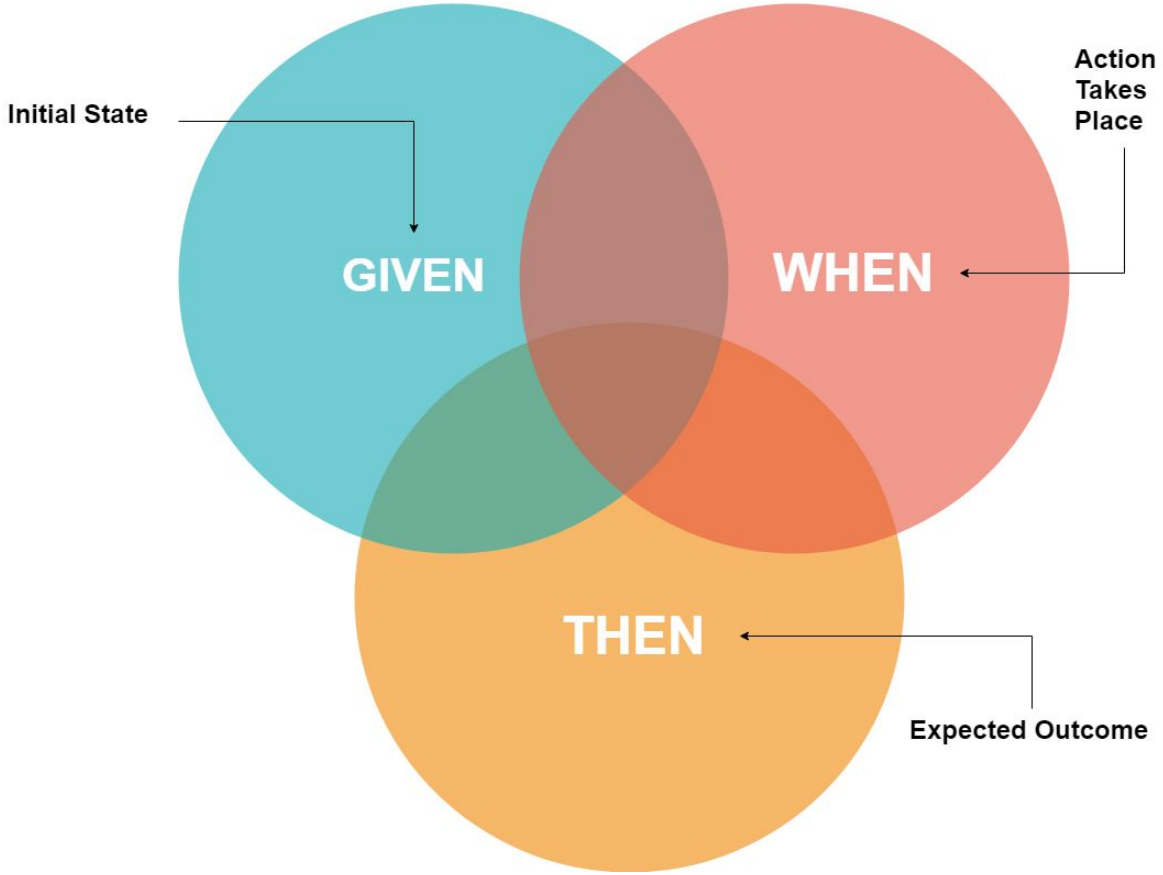


BDD

Behavior Driven Development:

1. Аналитик пишет UserStory с использованием ключевых слов
2. Разработчик пишет фреймворк с привязкой ключевых слов к коду
3. Запускаем тестирование кода с требованием аналитика

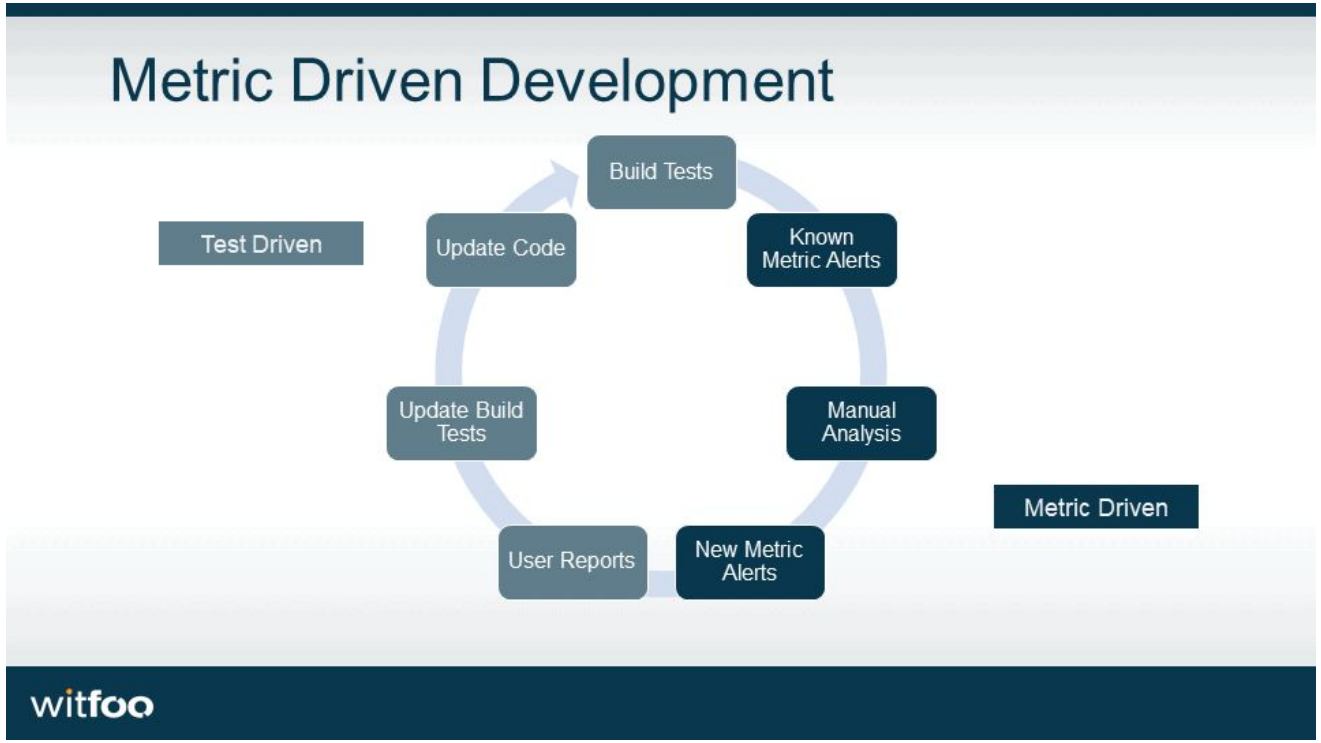
Дополнительные требования для аналитиков, дополнительная прослойка для тестирования, тяжелая поддержка



MDD

Metrics Driven Development:

1. Формируем бизнес-требования к системе
2. Формализуем требования в физические параметры, создаем перечень метрик
3. Формируем тестовые кейсы - действие+метрика
4. Кейсы “красные”
5. Реализуем логику, добиваемся “зеленых” тестов
6. Оптимизируем/рефакторим код, поддерживаем “зеленые” тесты

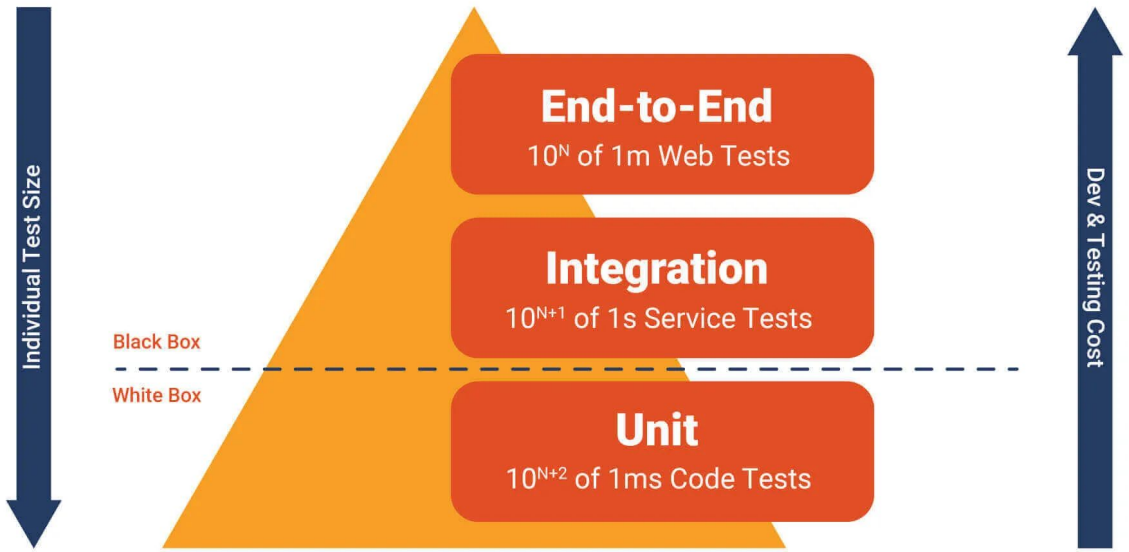


Дополнения по архитектуре

Дополнение к архитектуре

Требования:

1. Различные уровни тестирования:
   a. модульное
   b. интеграционное
   c. сквозное
2. Инфраструктура для генерации метрик
3. Специальный режим тестирования в приложении:
   a. объект debug в запросах и ответах
4. Каждый запрос содержит идентификатор для отслеживания всего пути его через систему
   a. По стандарту в заголовках HTTP



Инструменты тестирования

Testing Frameworks

Фреймворки:

1. Kotlin.test (https://kotlinlang.org/api/latest/kotlin.test/)
   a. test
   b. test-js
   c. test-junit
   d. test-junit5
   e. test-testing (TestNG)
2. Kotest (https://github.com/kotest/kotest)
3. Spek (https://www.baeldung.com/kotlin/spek)

Testing Frameworks

Инструменты для тестирования:

1. MockK (https://mockk.io/)
2. Mockito (https://site.mockito.org/)
3. Mockative (https://github.com/mockative/mockative)
4. TestContainers (https://www.testcontainers.org/)
5. Cucumber (https://cucumber.io/) - BDD
6. OpenSearch-stack (https://opensearch.org/) - тестирование и мониторинг

LIVE

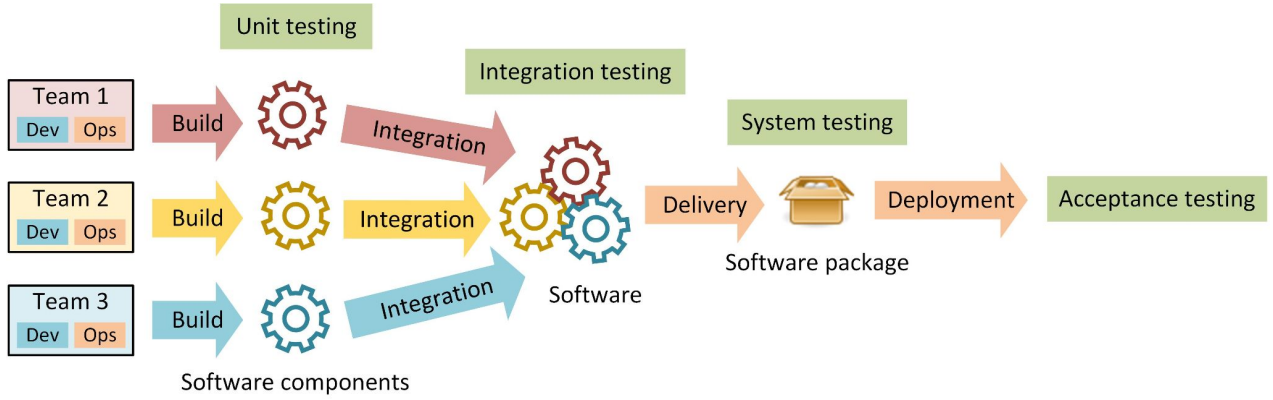
test frameworks

Интеграционные и сквозные тесты

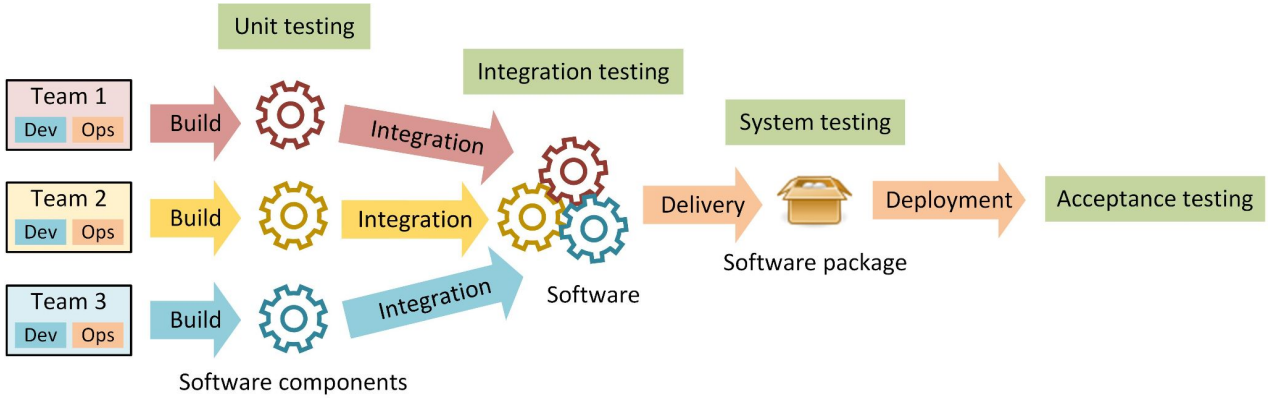
Интеграционные и сквозные тесты

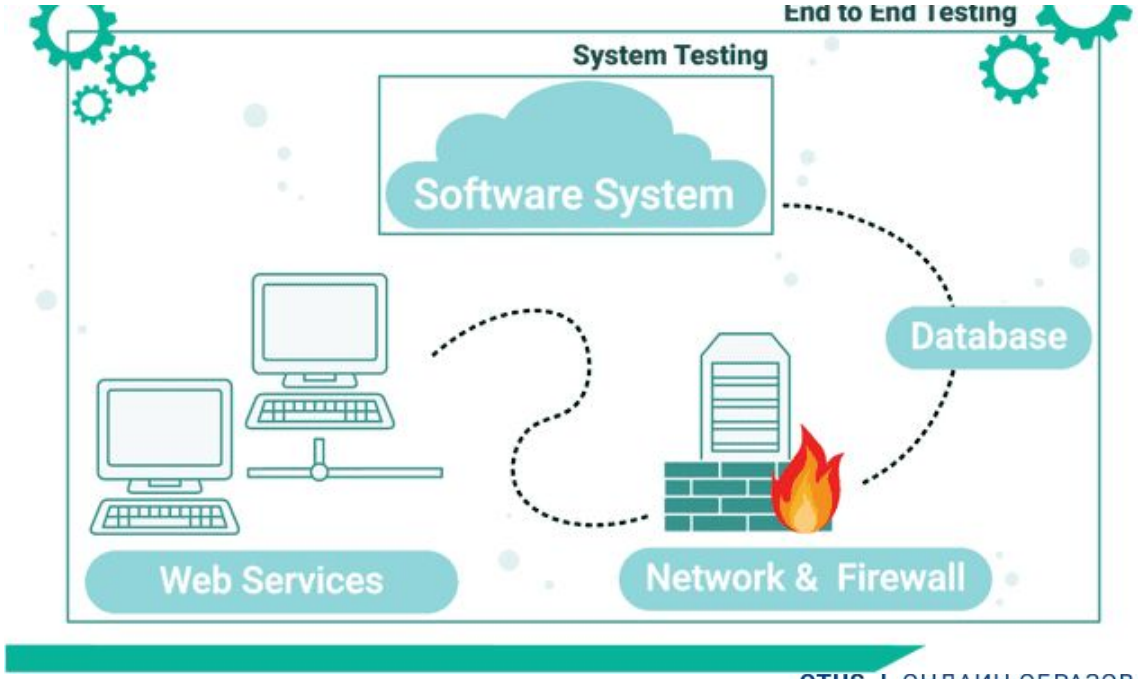
<u>Интеграционные</u>:

Тестирование конкретных интеграций



Интеграционные и сквозные тесты





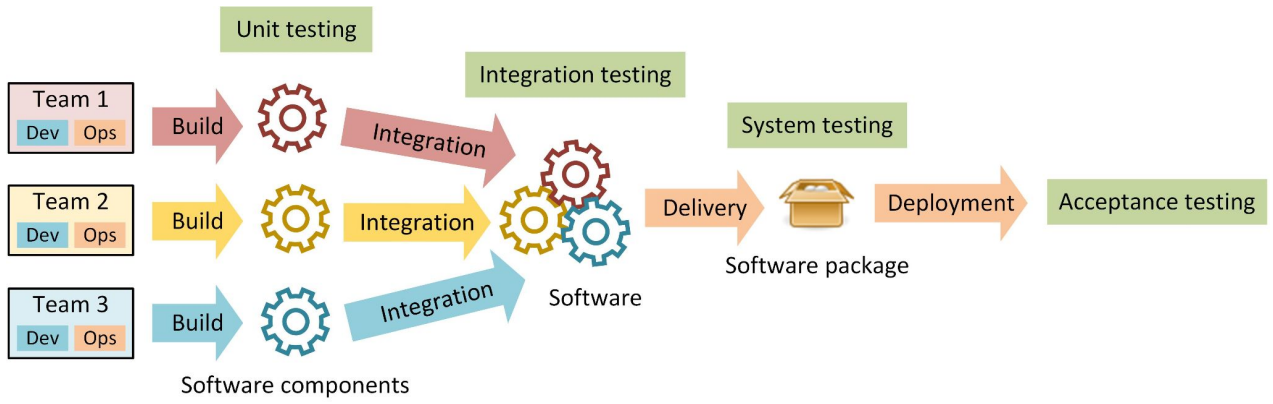
Сквозные:

1. Проверки всей системы в сборе
2. Запуск всей инфраструктуры в целом
3. Тестирование через фронтенд, либо коммерческие API

Интеграционные и сквозные тесты

<u>Общее</u>:

1. Считается, что дорогие, их следует уменьшать
2. Используем генераторы данных
3. Практика: зависит от реализации в проекте
4. Практика: аналитики могут давать задание в виде готовых тестов



LIVE

сквозные тесты

Вопросы?

Слайд с заданием

1. Какие бывают уровни тестирования?
2. Какие виды тестирования можете вспомнить?
3. Как происходит разработка в TDD и MDD?

Тезисы

Подведем итоги

1. Уровни тестирования: модульное, интеграционное, системное/сквозное, приемочное
2. Виды: функциональное, нефункциональное, изменений
3. TDD - красный тест, разработка/зеленение, рефакторинг
4. Тестовые фреймворки: kotlin.test, kotest
5. Инструменты: MockK, TestContainers
6. Сквозное тестирование через TestContainers

Слайд с домашним заданием

1. Сформировать транспортные модели для вашего выпускного проекта
2. Сделать тест на сериализацию/десериализацию
3. Реализовать внутренние модели
4. Обеспечить маппинг между транспортными и внутренними моделями
5.

Сроки выполнения: **19.08.2025**

Рефлексия

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Цели вебинара

Проверка достижения целей

1. Понимать назначение различных типов тестирования и планировать их в проектах
2. Использовать различные тестовые фреймворки и инструменты
3. Разрабатывать в парадигме TDD и учитывать MDD

Следующий вебинар

08.08.2025

Модуль 4: Разработка API

Мониторинг, сбор логов и метрик

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://kotlinlang.org/
2. https://gradle.org/
3. https://github.com/otuskotlin/202505-ok-marketplace
4. https://habr.com/ru/post/279535/
5. https://blog.jetbrains.com/ru/youtrack/2020/11/make-it-workflow-part-13-supporting-test-management-scenarios/
6. https://kotlinlang.org/api/latest/kotlin.test/
7. https://mockk.io/
8. https://www.testcontainers.org/
9. https://www.spekframework.org/
10. https://github.com/kotest/kotest
11. Allure https://qameta.io/, https://docs.qameta.io/allure/
12. https://wiremock.org/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары